

TOPIC NAME: Word Embedding

DAY: _____

TIME: _____

DATE: / /

Neural Networks understand numbers. So, we must convert words into numbers. But we can't use random numbers.

Example: great $\rightarrow$ 4.3
awesome $\rightarrow$ -32.1

Even though great and awesome mean similar things, their numbers are very different. So, here learning about "great" doesn't help the model understand "awesome."

So what is word embedding?

Represent each word as numbers following -

- i) similar words $\rightarrow$ similar numbers
 - ii) Words used in similar contexts $\rightarrow$ close together
- These numbers are learned, not assigned manually.

Why multiple numbers for one word?

A word can have different meanings depending on context.

Word	Embedding 1 (positive usage)	Embedding 2 (negative usage)
Great	1.85	0.98
Awesome	1.81	0.99

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

Positive: "Azmar is great"

Negative: "My phone is broken, great."

So, instead of one number, we use multiple numbers for "great" in different context.

Training Example:

Training Data:

Troll 2 is great!

Gymkata is great!

Here: "Troll 2" and "Gymkata" appear in the same context. So their embeddings should become similar.

Step 1: Create Inputs (One Hot Encoding)

Unique words: Troll 2, is, great!, Gymkata

Each word becomes an input neuron.

One hot encoding: Only one input is 1, others are 0.

Example (for "Troll 2"):

Troll 2 $\rightarrow 1$

is $\rightarrow 0$

great! $\rightarrow 0$

Gymkata $\rightarrow 0$

} This is called one hot encoding

GOOD LUCK

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

Step 2: Hidden Layer = Word Embeddings

- i) Inputs connect to hidden neurons
- ii) These neurons use identity activation
- iii) The weights of these connections are the word embeddings.

If we use 2 hidden neurons, each word gets: 2 embedding values

These weights start random and improve through training.

Step 3: Predict the Next Word

Each word tries to predict the next word.

Examples:

Troll 2 → predicts "is"

is → predicts "great!"

Gymkata → predicts "is"

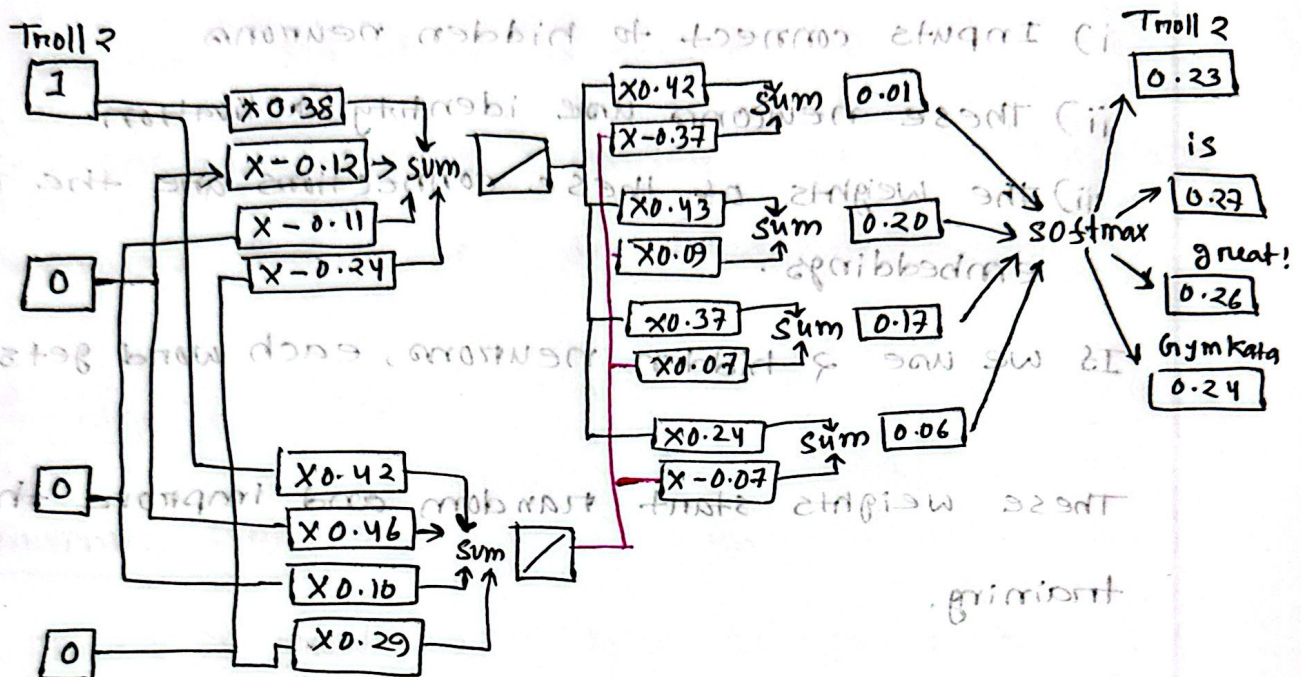
The network outputs probabilities using softmax.

TOPIC NAME : _____

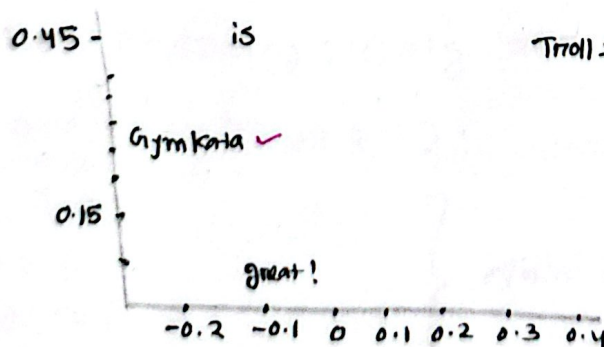
DAY : _____

TIME : _____

DATE : / /



Thus, using the untrained weights, putting a 1 in the input for Troll 2 and 0s in all of the other inputs. That correctly predicts the next word, "is", because it has the largest softmax value, 0.27, but just barely. But predictions are mostly wrong here, word embeddings are random and similar words are not close in embedding space!



Troll 2 ✓

Troll 2 and Gymkata are no more similar to each other.

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

However, because both words appear in the same context in the training data, optimizing the Word Embeddings with backpropagation should make their weights more similar.

Training with Backpropagation:

- i) Use cross-entropy loss
- ii) Adjust weights using Backpropagation
- iii) Words appearing in similar contexts move closer together.

After training:

"Troll 2" and "bjmktata" become near each other and model predicts correct next words.

This is the key success of word embedding.

* We don't care about the final outputs, we care about the learned weights (embeddings).

Word2Vec: Adding more context

Predicting just the next word is limited.

So Word2Vec uses two methods.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

i) Continuous Bag of Words (CBOW)

Uses surrounding words to predict the middle word.

Example : Troll 2 — great!

Prediction : is

ii) Skip Gram

Uses the middle word to predict surrounding words

Example: Input: is

Output: Troll 2, great!, Gymkata

Skip Gram works very well for rare words.

Large Scale Training:

i) use millions of sentences

ii) Example dataset: entire Wikipedia

iii) Vocabulary: 10000+ words

iv) Embedding size: 100 - 300

Training is slow with full softmax.

Negative Sampling (Speed Boost) :

Instead of updating all outputs:

Update only: correct word (target = 1), few random

wrong words (target = 0)

Much faster training

Almost same accuracy